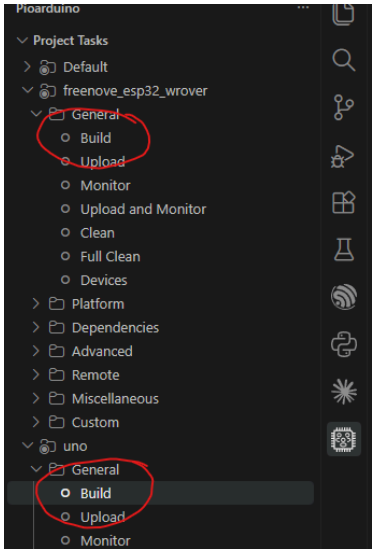
CEG 4330/6330 – Microprocessor-based Embedded Systems

Project 4 – PWM and Timers

Overview

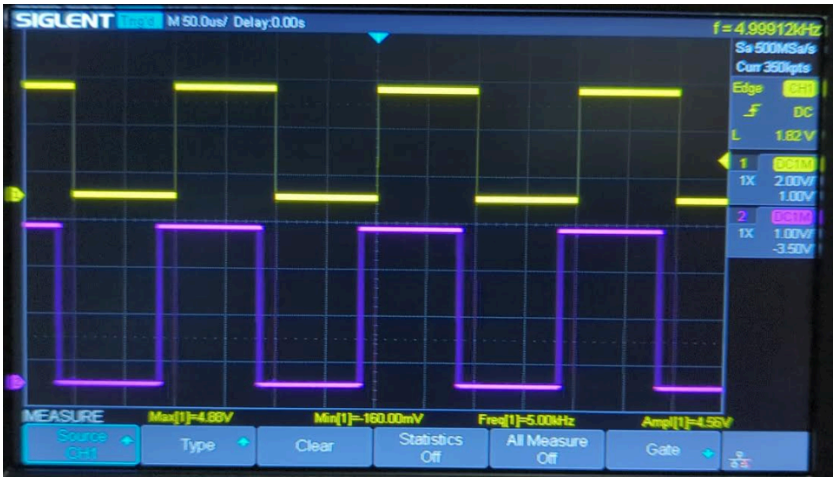
In this project, you are going to get familiar with the usage of PWM and timers on the ESP32 and Arduino Uno. Luckily, code for both platforms can be written with the same libraries and extensions we have already been using. Also, if you have multiple group members, you can split the work across platforms. Please use the provided starter code to get started.

NOTE: Both programs can be built, uploaded, and monitored from the same pioarduino project on the same computer:



PWM Output

Using the sample code, build and upload both to their respective boards. Afterwards, display each PWM signal on a different channel on the oscilloscope and measure the duty cycle, frequency, and max voltage of both. Make note of how both platforms allow for changing frequency and duty cycle. On ESP32 there are the ledc functions but on the Arduino it requires manual register manipulation. Also, note that the Arduino outputs 5V where the ESP32 outputs 3.3V:



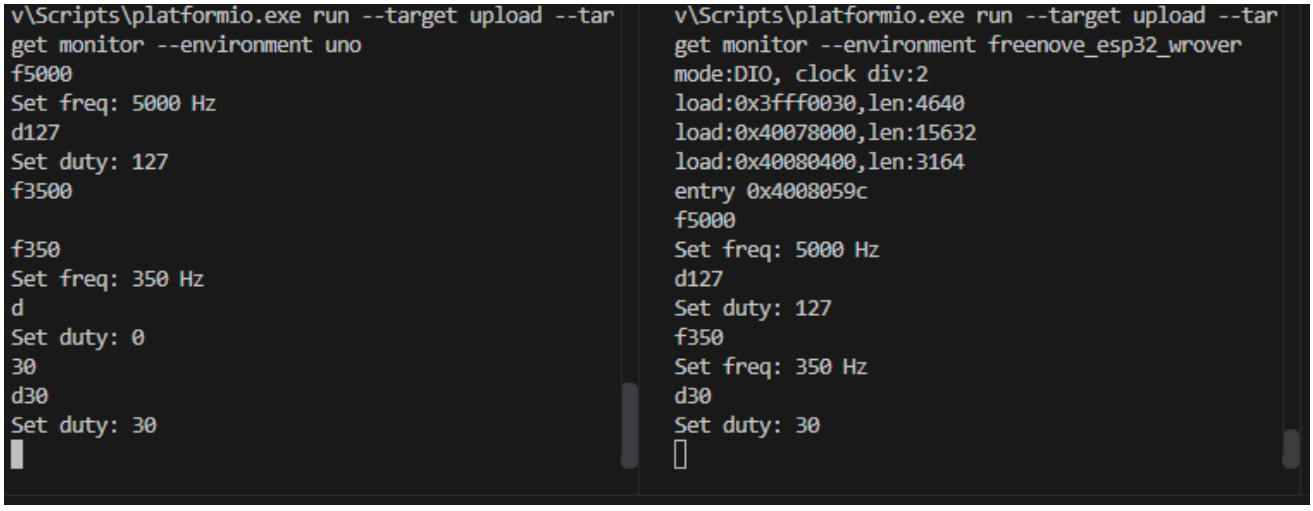
(Yellow: Arduino) (Purple: ESP32)

Display both of the PWM signals on the oscilloscope to the TA.

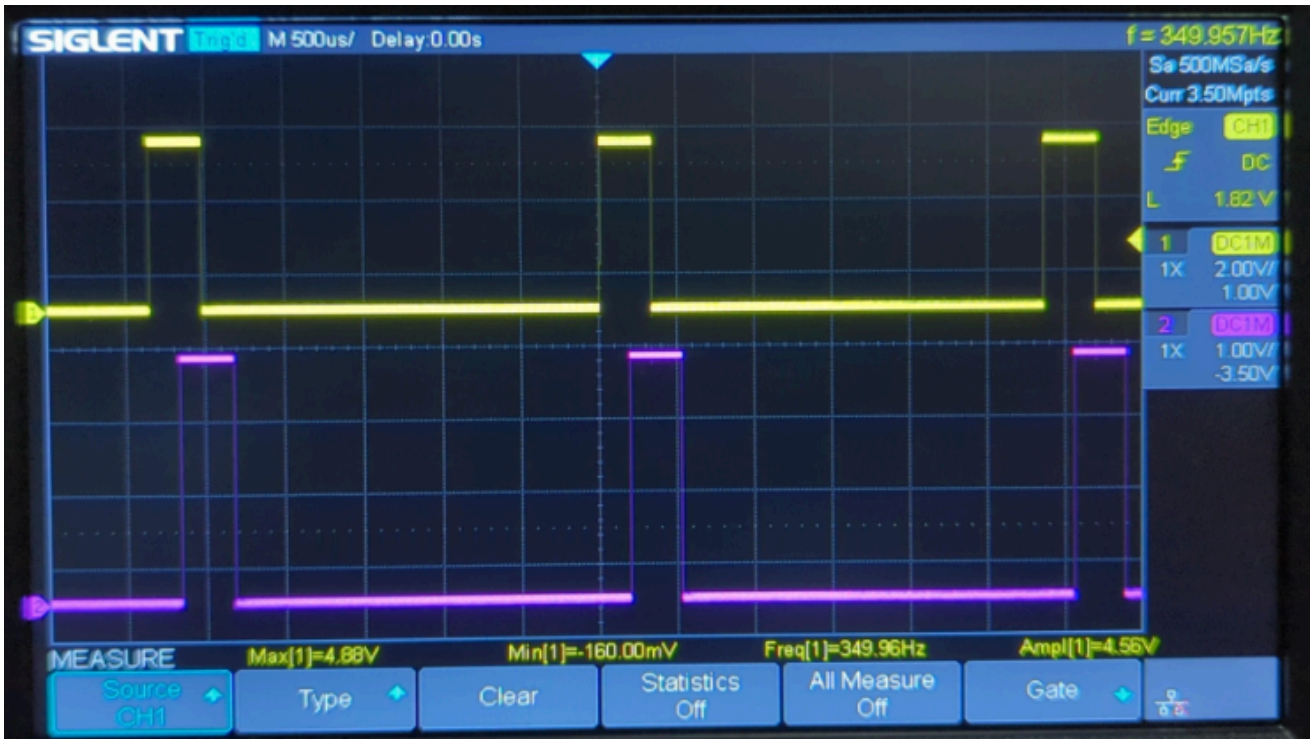
Setting Output via Serial Monitor

Next, implement controlling the duty cycle via the serial monitor on both platforms. Implement the following functions on both the ESP32 and Arduino code:

- void getSerialInput()
    - Check if Serial.available() (this function returns true if you send data to the dev board through the terminal)
        - If no serial is available, just return
    - If serial data is available you can read an entire string using Serial.readStringUntil('\n');
        - Hint: To get a substring use myString.substring(index);
        - Hint: A String can be converted to an integer by calling myString.toInt();
    - If the first character is a “d”
        - Pass the integer to the changeDutyCycle() function
    - If the first character is a “f”
        - Pass the integer to the changeFrequency() function
    - Hint: This will allow you to supply a “command” with “arguments” like “d 127” for 50% duty cycle or “f 1000” for 1000 Hz frequency
- void changeDutyCycle(long value)
    - Limit the value to be in the range of 0 to 255
    - Change the duty cycle
        - On ESP32, use the ledc interface
        - On Arduino, use timer registers
- void changeFrequency(long value)
    - Limit the value to be in the range of 100 to 10000
    - Change the frequency
        - On ESP32, use the ledc interface
        - On Arduino, use timer registers



Changing frequency and duty cycle. (Arduino on left, ESP32 on right)



(Yellow: Arduino) (Purple: ESP32)

Display changing the duty cycle and frequency of both of the PWM signals on the oscilloscope to the TA.

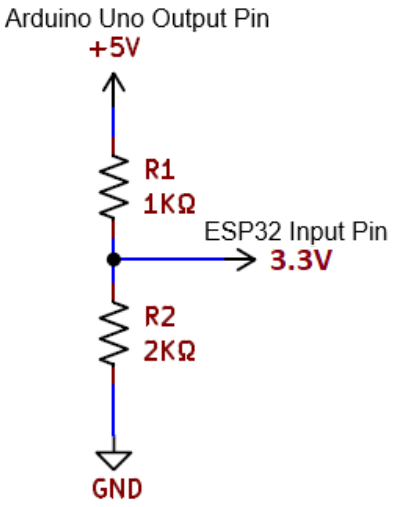
Note: due to the 0 to 255 range and timer resolution, achieving a perfect duty cycle and frequency is often impossible. Achieving a duty cycle or frequency within +/- 5% on the oscilloscope from what is typed in the serial monitor is acceptable for the remainder of this lab.

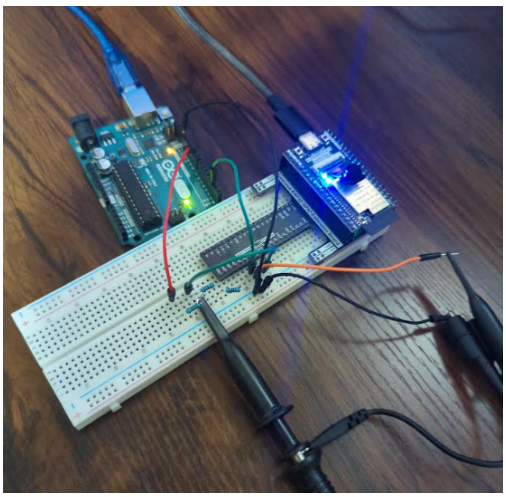
Displaying Output via Serial Monitor

Next, implement reading a PWM input for both devices, i.e. ESP32 reads the PWM output from the Arduino and Arduino reads the PWM output from the ESP32.

NOTE: The Arduino GPIO is 5V and ESP32 GPIO is 3.3V. If you directly connect an Arduino output pin to the ESP32 you could destroy the ESP32.

Use a voltage divider to reduce the voltage from 5V to 3.3V:



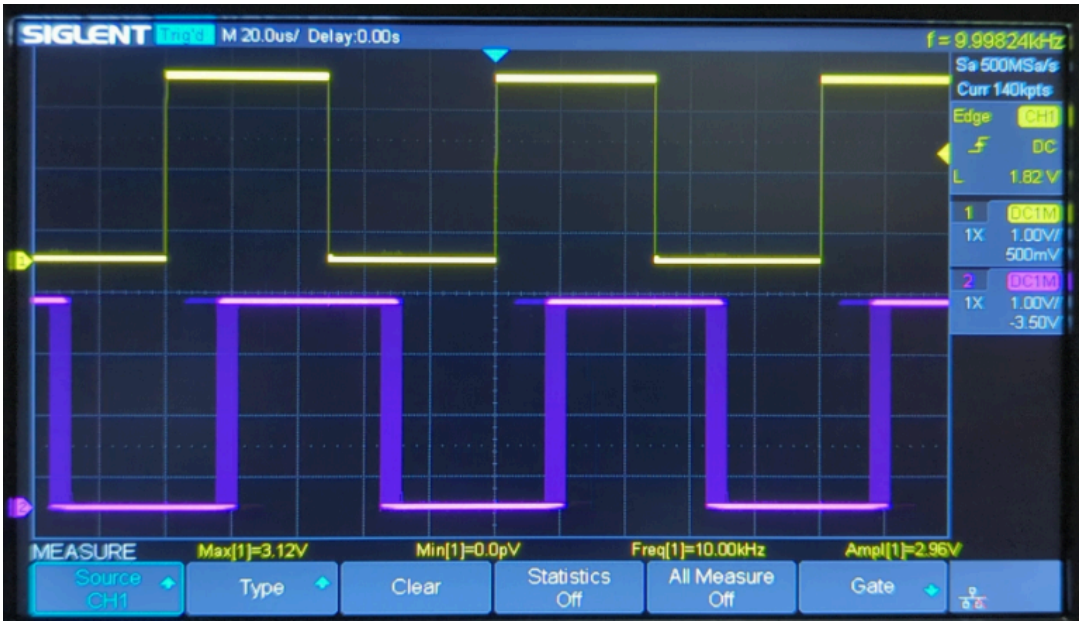


On both the ESP32 and the Arduino, write an ISR that measures the frequency and duty cycle of the incoming PWM signal. Then, implement the displayIncomingPWM() function on both the ESP32 and the Arduino Uno. Make sure to limit the speed that you write to the serial monitor. Writing to the serial monitor once per second is suitable.

```
v\Scripts\platformio.exe run --target upload --tar
get monitor --environment uno
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 52.0 %
Freq: 10000 Hz, Duty: 52.0 %
Freq: 10000 Hz, Duty: 52.0 %
Freq: 10000 Hz, Duty: 52.0 %
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 48.0 %
Freq: 10000 Hz, Duty: 52.0 %
Freq: 10000 Hz, Duty: 48.0 %
```

```
v\Scripts\platformio.exe run --target upload --tar
get monitor --environment freenove_esp32_wrover
Freq: 10204 Hz, Duty: 50.0 %
Freq: 9901 Hz, Duty: 48.5 %
Freq: 9901 Hz, Duty: 48.5 %
Freq: 10000 Hz, Duty: 50.0 %
Freq: 10000 Hz, Duty: 50.0 %
Freq: 10101 Hz, Duty: 50.5 %
Freq: 10101 Hz, Duty: 50.5 %
Freq: 10000 Hz, Duty: 50.0 %
Freq: 9901 Hz, Duty: 48.5 %
Freq: 10101 Hz, Duty: 49.5 %
Freq: 10101 Hz, Duty: 48.5 %
Freq: 9901 Hz, Duty: 46.5 %
Freq: 10101 Hz, Duty: 49.5 %
```

Displaying incoming frequency and duty cycle. (Arduino on left, ESP32 on right)
(Notice the frequency and duty cycle jitter)



(Yellow: Arduino) (Purple: ESP32)

Display reading the duty cycle and frequency on the Arduino Uno's and ESP32's serial monitors and the PWM signals on the oscilloscope to the TA. Change the frequency and duty cycle of both devices and show the correct reading on the serial monitor and oscilloscope.

Graduate Students CEG 6330 Only:
Replace the input code on the Arduino Uno to utilize input capture to measure the PWM duty cycle and frequency. Implementing input capture makes measuring the incoming PWM signal much more accurate especially at higher frequencies.

Display the input capture code and display reading the duty cycle and frequency on the Arduino Uno's serial monitor and the PWM signals on the oscilloscope to the TA.

How to Submit

Demonstrate the various milestones mentioned above to the TA in person. Upload a link to your GitHub repo to the Pilot Project dropbox.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
Displayed ESP32 and Arduino Uno PWM Output	1.00
Implemented setting PWM via serial monitor on ESP32	1.00
Implemented setting PWM via serial monitor on Arduino Uno	1.00
Implemented reading PWM from Arduino Uno into ESP32	1.00
Implemented reading PWM from ESP32 into Arduino Uno	1.00
Implemented input capture for Arduino PWM input (CEG 6330 students only)	20%
Total	5.00